



Progetto MCF

Federico Manni

RICHIESTA

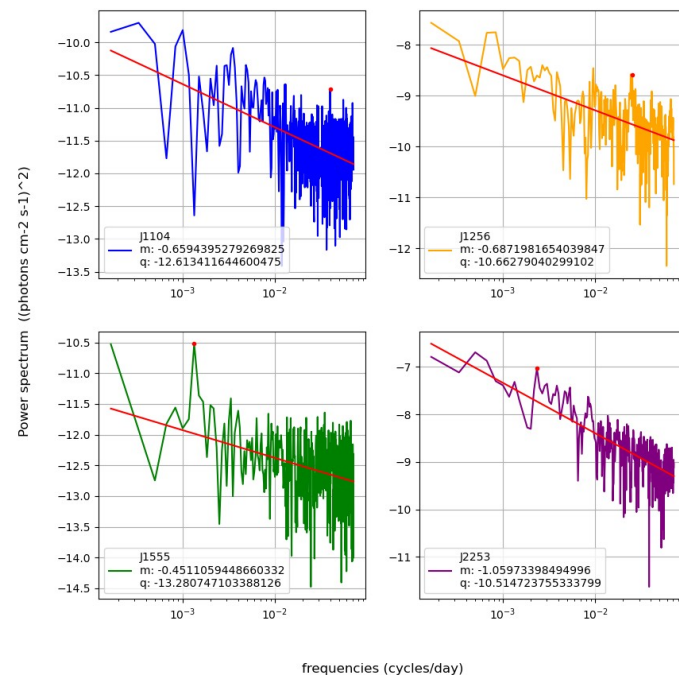
La richiesta è quella di effettuare l'analisi delle curve di luce prodotte da 4 blazar, i cui dati sono raccolti dal Fermi-LAT su base settimanale e mensile, al fine di individuare segnali di periodicità nello spettro di potenza.

STRATEGIA

- Si effettua la trasformata di Fourier e si calcola lo spettro di potenza della curva di luce.
- Ricerca dei picchi particolarmente significativi nello spettro di potenza:
 - Ciò viene fatto calcolando la differenza tra i punti dello spettro in scala logaritmica e la retta di fit dello spettro, sempre in scala logaritmica, che dovrebbe essere il modello.
 - La scala log. permette di gestire meglio le grandi differenze dei valori dello spettro
 - Individuazione del massimo valore di tutte queste differenze

- Generazione di molte SLC a partire dalla curva originale
- Ricerca picchi più significativi per ogni SLC generata
- Calcolo probabilità che un picco nella SLC disti dal proprio modello più del picco nella curva originale. $:= (\text{prob})$
- La significatività è data come $1 - \text{prob}$

Spettro di potenza (loglog) & fit, campionamento settimanale



CRITICITÀ

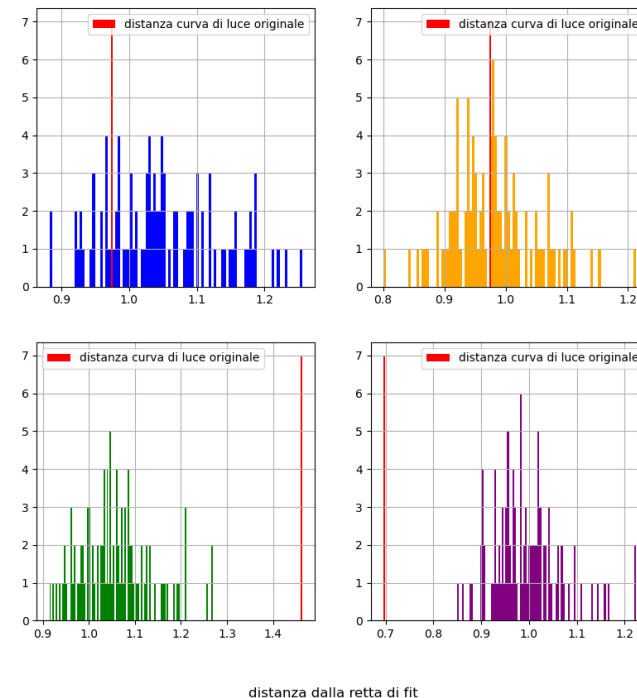
I Blazar hanno un rumore rosa mentre lo shuffle che si esegue rompe la correlazione producendo curve con un rumore bianco.

- Rischio di sovrastimare la significatività alle basse frequenze

IL CODICE

STRUTTURA DATI

- Creazione di una classe per le sorgenti i cui attributi sono le grandezze utili calcolate. Le istanze della classe sono poi salvate in un dizionario.
- Gli attributi sono dei dizionari in cui sono presenti la stessa grandezza per la LC a campionamento mensile o settimanale.



distanza dalla retta di fit

```
self.pf = {'W' : corr_pf(file_path_W).to_numpy(dtype = float), 'M' : corr_pf(file_path_M).to_numpy(dtype = float)}
self.Ft = {'W' : Ft(self.pf['W']), 'M': Ft(self.pf['M'])}
self.ps = {'W': np.abs(self.Ft['W'])**2, 'M': np.abs(self.Ft['M'])**2}
self.logps = {'W': np.log10(self.ps['W'])[1:], 'M': np.log10(self.ps['M'])[1:]}
self.frq = {'W': fft.rfftfreq(len(self.pf['W']), d = 7), 'M' : fft.rfftfreq(len(self.pf['M']), d = 30)}
```

- Per le curve di luce sintetiche non si sono usate strutture dati

- l'elevato numero di simulazioni rende inefficiente allocare e distruggere queste strutture.

FUNZIONI

- Nel codice si fa uso di funzioni per svolgere compiti che richiederebbero più righe e che vanno ripetuti sia per le curve di luce originali che per le SLC

```
def fit(logps, frq, F_fit, p0):  
    """  
    Esegue: fit con una retta in scala loglog dello spettro di potenza  
    Restituisce: Punti calcolati in corrispondenza delle frequenze sulla curva fittata e i parametri  
  
    Input:  
  
        # F_fit -> funzione per il fit  
        # p0 -> parametri iniziali per il fit:  
        #     [0]: pendenza; [1]: intercetta  
  
    """  
    x = np.log10(frq[1:]) #il valore per [0] è zero  
    params, pcov = optimize.curve_fit(F_fit, x, logps, p0)  
    y_fit = F_fit(x, params[0], params[1]) #una retta in scala loglog sarebbe del tipo  $\log(y) = m\log(x)$   
  
    return y_fit, params
```

PARSER

- La generazione di grafici è subordinata all'uso di righe di comando da terminale
- Potendone generare molteplici, mandarli tutti a schermo, sempre, sarebbe risultato pesante e confusionario.

```
if args.PhotonFluxW:  
    fig1, pfgW = plt.subplots(2,2)  
  
    fig1.suptitle('Curva di luce, campionamento settimanale')  
  
    fig1.supxlabel('time (Julian date)')  
    fig1.supylabel('Photon Flux [0.1-100 GeV](photons cm-2 s-1)')  
    for key,j,k in zip(Sorgenti, (0,0,1,1),(0,1,0,1)):  
        pfgW[j][k].plot(Sorgenti[key].JdW, Sorgenti[key].pf['W'], c = Sorgenti[key].color, label = key)  
        pfgW[j][k].grid(True)  
        pfgW[j][k].legend()
```

DETTAGLI DEL CODICE

-Si è cercato di usare quanto più possibile i metodi di numpy e pandas per effettuare le manipolazioni sugli array. Tutto è convertito in un array numpy

-L'eliminazione di '<' è avvenuta tramite metodo di 'Pandas' ".str.lstrip('<')": (Strip whitespaces (including newlines) or a set of specified characters from each string in the Series/Index from left side.)

-La trasformazione da Df a Series tramite .squeeze(). Non usarlo genera errore con .str.lstrip()

-La trasformazione da DataFrame a array 1d di numpy tramite avviene tramite .to_numpy() e .flatten(). Senza si avrebbe un array ndimensionale [... [..], [..] ...] che crea problemi con i metodi di scipy successivi

- La trasformata è eseguita con il metodo `fft.rfft()` di scipy poichè gli input sono tutti reali.

- Il valore a frequenza nulla è la componente continua del segnale. Appare come un picco nello spettro di potenza che schiaccia tutti gli altri valori, quindi è arbitrariamente posta a zero. Non influenza l'analisi

- Quando si lavora con i logaritmi gli zeri sono un problema. I primi punti degli array dello spettro e delle frequenze sono ignorati tramite slicing ([1:]). Essendo zero entrambi numpy restituirebbe -inf

-La generazione di SLC è problematica: può capitare che la FFT di una SLC restituisca uno zero a una qualsiasi frequenza

```

0.9741912970107274    0.827    236
0.9753742657503892    0.601    150
/home/federicomanni/Scrivania/Università/Informatica/MetodiComp/MCF/Progetto/Progetto-MCF/programma/analisi_dati2.py:231: RuntimeWarning: divide by zero encountered in log10
  slclogpsM = np.log10(slcpsM[1:])
Traceback (most recent call last):
  File "/home/federicomanni/Scrivania/Università/Informatica/MetodiComp/MCF/Progetto/Progetto-MCF/programma/analisi_dati2.py", line 233, in <module>
    slclog_fitW, slcparamsW, slclog_fitM, slcparamsM = fit(slclogpsW, slclogpsM, key.frcW, key.frcM, F_fit, key.p0)
  File "/home/federicomanni/Scrivania/Università/Informatica/MetodiComp/MCF/Progetto/Progetto-MCF/programma/analisi_dati2.py", line 96, in fit
    paramsM, pconvM = optimize.curve_fit(F_fit, xM, logpsM, p0)
  File "/usr/lib/python3/dist-packages/scipy/optimize/_minpack.py", line 735, in curve_fit
    ydata = np.asarray_chkfinite(ydata, float)
  File "/usr/lib/python3/dist-packages/numpy/lib/function_base.py", line 488, in asarray_chkfinite
    raise ValueError(
ValueError: array must not contain infs or NaNs

```

-Per controllare la presenza di zeri si usano insieme una maschera e il metodo di np.any()

-La maschera restituisce True in corrispondenza degli zeri dell'array

-np.any() restituisce un booleano True se rileva almeno un True nell'array, False altrimenti

-Se ciò accade la Ft va scartata e si deve ricominciare.

- ==0.0 vale anche se il numero è complesso

-Il numero di iterazioni può essere superiore a quello inizialmente previsto, quindi si usa un ciclo while. Il contatore di cicli eseguiti aumenta solo se il ciclo va a buon fine

```

while it < 1000: # dovendo saltare alcu

    slcW = np.copy(sorg.pf[W])
    slcM = np.copy(sorg.pf[M])
    np.random.shuffle(slcW)
    np.random.shuffle(slcM)

    slcFtW = Ft(slcW)
    slcFtM = Ft(slcM)

    maskM = slcFtM[1:] == 0.0
    maskW = slcFtW[1:] == 0.0
    if np.any(maskW) or np.any(maskM):
        continue

```

- senza `np.copy()` python crea un riferimento all'array originale, quindi sarebbe modificato quello.

- Potrei creare le copie prima del ciclo while e poi eseguire lo shuffle sempre della stessa. Il codice sarebbe più leggero ma ci sarebbe una correlazione tra le SLC credo.

- I parametri iniziali del fit, `p0`, sono uguali per tutti gli spettri. Se `scipy` non riesce a ottimizzarli restituisce un errore, cosa che non avviene.

- Li si può indovinare graficamente plottando uno spettro di potenza

- I grafici sono costruiti sfruttando un ciclo for nel dizionario delle istanze e il metodo "`zip()`". Questo permette una corretta disposizione dei grafici all'interno della fig.

- Un riassunto dei dati è mandato a schermo ricostruendo un `DataFrame Pandas` con il metodo `pd.DataFrame()`. Prende in input un dizionario ed associa ad ogni chiave una colonna e al rispettivo valore le righe.

Risultati

- Per il blazar `4FGL_J1555.7+1111` è emerso un picco molto significativo:

- dall'analisi su campionamento settimanale alla frequenza di `0.001338` cicli/giorno la significatività è 1 sempre, anche rimandando il codice più volte: non c'è mai stato un picco altrettanto significativo in nessuna slc

- dall'analisi su campionamento mensile alla frequenza di `0.001335` cicli/giorno circa, la significatività è circa `0,990`. Cambia di poco mandando il codice diverse volte se non si fissa il seed.

Poiché la significatività è molto alta e poiché le frequenze per le due analisi sono molto simili, questo risultato potrebbe essere un indice di periodicità. Va però considerato quanto detto all'inizio: le slc generate con un random shuffle potrebbero portare ad una sovrastima per i valori per le basse frequenze (indice 8 nel nostro caso)